

1) For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

Ans:

Given:

STUDENT_COURSE(SID, S Name, CID, C Name, Instructor, Grade)

1. Given Relation

STUDENT_COURSE(SID, S Name, CID, C Name, Instructor, Grade)

Where:

- $SID \rightarrow$ Student ID
- $S\ Name \rightarrow$ Student Name
- $CID \rightarrow$ Course ID
- $C\ Name \rightarrow$ Course Name
- $Instructor \rightarrow$ Course Instructor
- $Grade \rightarrow$ Grade obtained by the student

2. Functional Dependencies

We assume the following relationships:

FD 1: $SID \rightarrow S\ Name$

A Student ID uniquely identifies a student's name.

FD 2: $CID \rightarrow C\ Name$

A Course ID uniquely identifies the course name.

FD 3: $CID \rightarrow Instructor$

A Course ID identifies the instructor teaching that course.

FD 4: $(SID, CID) \rightarrow Grade$

A student's grade is determined by the combination of Student ID and Course ID.

3. Finding Candidate Key

We need to find an attribute or combination of attributes that can determine all attributes.

Consider SID alone:

$SID \rightarrow S\ Name$

But SID cannot determine CID, C Name, Instructor, or Grade.

So, SID is not a candidate key.

Consider CID alone:

$CID \rightarrow C\ Name, Instructor$

But CID cannot determine SID, S Name, or Grade.

So, CID is not a candidate key.

Now consider (SID, CID) :

$(SID, CID) \rightarrow S\ Name, C\ Name, Instructor, Grade$

Because:

- SID gives S Name
- CID gives C Name
- CID gives Instructor
- SID + CID gives Grade

Therefore:

Candidate Key = {SID, CID}

It is also the primary key candidate because together they uniquely identify each student-course enrollment.

4. Why is it a Composite Key?

The key contains two attributes:

SID + CID

Therefore, it is called a composite candidate key.

Functional Dependencies:

- $SID \rightarrow S\ Name$
- $CID \rightarrow C\ Name$
- $CID \rightarrow Instructor$
- $(SID, CID) \rightarrow Grade$

Candidate Key:

{SID, CID}

Type of Key: Composite Candidate Key

Reason: SID identifies the student, CID identifies the course, and together they uniquely identify a student's enrollment and grade.

2) Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Ans:

1. What is 1NF?

A relation is in First Normal Form (1NF) when:

- Each cell contains only one atomic value.

- There are no repeating groups or multi valued attributes.
- Each row can be uniquely identified.

2. Un normalized Relation

Consider the following **STUDENT** relation:

SID	S Name	Course	Grades
S01	ARJUN	DBMS,JAVA	A,B
S02	MEERA	PYTHON,JAVA.OS	A,B,A
S03	AADHYA	DBMS	A

Here, Courses and Grades contain multiple values in a single cell.

For example:

S02 → Python, Java, OS

This violates the atomicity rule.

Therefore, the relation is not in 1NF.

3. Identify the Problem

The following attributes contain multiple values:

- Courses
- Grades

This creates a repeating group.

4. Convert into 1NF

To convert it into 1NF, we separate each course and grade into individual rows.

STUDENT_COURSE in 1NF

SID	SNAME	COURSE	GRADE
S01	CHAITHU	DBMS	A
S01	ARUNA	JAVA	B
S02	GAYATHRI	PYTHON	A
S02	KUNDHANA	JAVA	B
S02	DEEKSHITHA	DBMS	A

5. Key Identification

A student can take multiple courses, so SID alone cannot uniquely identify a row.

Therefore, we use:

(SID, Course) as the composite key.

6. Justification

The converted relation is in 1NF because:

1. Every attribute contains atomic values.
2. There are no repeating groups.
3. Each row represents one student-course combination.
4. The composite key (SID, Course) uniquely identifies each record.

Un normalized:

STUDENT (SID, S Name, Courses, Grades)

Remove multi valued/repeating attributes

1NF:

STUDENT_COURSE (SID, S Name, Course, Grade)

Primary/Composite Key: (SID, Course)

3) **Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.**

Ans:

Decomposition of R into 2NF

Relation:

R(A,B,C,D,E)

Functional Dependencies (FDs):

1. $A \rightarrow BC$
2. $BC \rightarrow D$
3. $D \rightarrow E$

Step 1: Find the Candidate Key

From:

$A \rightarrow B, C$

and

$BC \rightarrow D$

and

$D \rightarrow E$

we get:

$A^+ = \{A, B, C, D, E\}$

Therefore, A determines all attributes.

So:

Candidate Key = A

Since the key has only one attribute, it is a single-attribute key.

Step 2: Check for Partial Dependency

A partial dependency occurs when a non-key attribute depends on only a part of a composite key.

Here, the candidate key is only A.

Therefore, there is no partial dependency.

For example:

- $A \rightarrow B, C$
- $A \rightarrow D$ (through BC)
- $A \rightarrow E$ (through D)

All non-key attributes ultimately depend on the complete key A.

Therefore:

$R(A, B, C, D, E)$ is already in 2NF.

There is no need to decompose the relation for 2NF.

Step 3: Important Observation

Although there is no partial dependency, there is a transitive dependency:

$A \rightarrow D$

and

$D \rightarrow E$

Therefore:

$A \rightarrow E$

This is a transitive dependency, which is relevant to 3NF, not 2NF.

4) Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

Solu:

Given Relation

EMPLOYEE(Emp ID, Emp Name, DeptI D, Dept Name, Manager ID)

Assume the following Functional Dependencies:

1. $\text{Emp ID} \rightarrow \text{Emp Name, Dept ID}$
2. $\text{Dept ID} \rightarrow \text{Dept Name, Manager ID}$

Step 1: Find the Candidate Key

From:

$\text{Emp ID} \rightarrow \text{Emp Name, Dept ID}$

and

$\text{Dept D} \rightarrow \text{Dept Name, Manager ID}$

we can derive:

$\text{Emp ID} \rightarrow \text{Emp Name, Dept ID, Dept Name, Manager ID}$

Therefore:

Candidate Key = Emp ID

Step 2: Identify the Transitive Dependency

We have:

$\text{Emp ID} \rightarrow \text{Dept ID}$

and

$\text{Dept ID} \rightarrow \text{Dept Name, Manager ID}$

Therefore:

$\text{Emp ID} \rightarrow \text{Dept Name, Manager ID}$

This means Dept Name and Manager ID depend on Emp ID through Dept ID.

So, there is a transitive dependency:

$\text{Emp ID} \rightarrow \text{Dept ID} \rightarrow \text{Dept Name, Manager ID}$

This violates 3NF.

Step 3: Decompose the Relation

To remove the transitive dependency, divide the relation into two relations.

Relation 1: EMPLOYEE

EMPLOYEE(Emp ID, Emp Name, Dept ID)

Emp ID	Emp Name	Dept id
E01	Lavanya	D01
E02	Navya	D02
E03	Divya	D01

Primary Key: Emp ID

Relation 2: DEPARTMENT

DEPARTMENT(Dept ID, Dept Name, Manager ID)

Dept id	Dept name	Manager id
D01	CSE	M01

Step 4: Check 3NF

EMPLOYEE

$\text{EmpID} \rightarrow \text{EmpName, DeptID}$

The non-key attributes depend directly on the primary key.

EMPLOYEE is in 3NF.

DEPARTMENT

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

The non-key attributes depend directly on the primary key.

DEPARTMENT is in 3NF.

Primary Keys:

- $\text{EMPLOYEE} \rightarrow \text{EmpID}$
- $\text{DEPARTMENT} \rightarrow \text{DeptID}$

Foreign Key:

- $\text{EMPLOYEE.DeptID} \rightarrow \text{DEPARTMENT.DeptID}$

Conclusion

The original EMPLOYEE relation had a transitive dependency:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

By separating employee and department information into two relations, the transitive dependency is eliminated and the relations are brought into Third Normal Form (3NF).

5) Verify whether $R(A,B,C,D)$ with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

Given

$R(A, B, C, D)$

Functional Dependencies:

- $A \rightarrow BC$
- $C \rightarrow D$
- $D \rightarrow A$

Step 1: Find Candidate Keys

A^+

$A \rightarrow B, C$

$C \rightarrow D$

Therefore:

$A^+ = \{A, B, C, D\}$

So, A is a candidate key.

C^+

$C \rightarrow D$

$D \rightarrow A$

$A \rightarrow B, C$

Therefore:

$C^+ = \{A, B, C, D\}$

So, C is a candidate key.

D^+

$D \rightarrow A$

$A \rightarrow B, C$

Therefore:

$D^+ = \{A, B, C, D\}$

So, D is a candidate key.

Step 2: Check BCNF

A relation is in BCNF if:

For every FD $X \rightarrow Y$, X must be a superkey.

Check:

- $A \rightarrow BC \rightarrow A$ is a candidate key
- $C \rightarrow D \rightarrow C$ is a candidate key
- $D \rightarrow A \rightarrow D$ is a candidate key

Therefore, all determinants are candidate keys.

Final Answer

$R(A,B,C,D)$ is already in BCNF.

No decomposition is required.

6. Experimentally verify lossless-join decomposition using the Chase algorithm

To experimentally verify lossless-join decomposition using the Chase (tabular) algorithm, you can write the answer like this.

Suppose we have a relation:

$R(A,B,C)$

with the functional dependency:

$A \rightarrow B$

and it is decomposed into:

$R_1(A,B)$ $R_2(A,C)$

We need to check whether this decomposition is **lossless**.

Step 1: Construct the Chase Table

Use one row for each decomposed relation.

Relation **A** **B** **C**

$R_1(A,B)$ a_1 a_2 b_{13}

$R_2(A,C)$ a_1 b_{22} a_3

Here, a_i is used when the attribute is present in that relation, while b_{ij} represents a distinct value when the attribute is absent.

Step 2: Apply the Functional Dependency

Given:

$A \rightarrow B$

Both rows have the same value a_1 for attribute **A**.

Therefore, according to the FD $A \rightarrow B$, their **B values must also become equal**.

Since the first row already contains the distinguished symbol a_2 , replace b_{22} with a_2 .

The table becomes:

Relation	A	B	C
R1	a_1	a_2	b_{13}
R2	a_1	a_2	a_3

Step 3: Check for a Complete Distinguished Row

The second row now contains:

(a_1, a_2, a_3)

That means one row consists entirely of distinguished a-symbols.

Therefore, according to the Chase test:

The decomposition is lossless-join

Conclusion

The decomposition

$R(A, B, C) \rightarrow R_1(A, B), R_2(A, C)$

is **lossless** under the functional dependency

$A \rightarrow B$

because after applying the Chase algorithm, one row becomes completely composed of distinguished symbols.

For an exam, the key line is:

If any row becomes entirely a-symbols, the decomposition is lossless.

Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

Consider the relation:

TEACHER(TID, Subject, Hobby)

Assume a teacher can teach multiple subjects and can have multiple hobbies, and these two sets are independent.

The multivalued dependencies are:

$TID \twoheadrightarrow Subject$

and

$TID \twoheadrightarrow Hobby$

This means that for a particular teacher, the subjects they teach do not depend on their hobbies, and vice versa.

The relation is not in 4NF because TID is not a superkey of the original relation, yet it determines independent multivalued attributes.

So decompose it into:

TEACHER_SUBJECT(TID, Subject)

and

TEACHER_HOBBY(TID, Hobby)

Example

Original relation:

TID	Subject	Hobby
T1	DBMS	Cricket
T1	DBMS	Music
T1	Java	Cricket
T1	Java	Music

This creates unnecessary combinations between subjects and hobbies.

After 4NF decomposition:

TEACHER_SUBJECT

TID Subject

T1 DBMS

T1 Java

TEACHER_HOBBY

TID Hobby

T1 Cricket

T1 Music

Therefore:

$TID \twoheadrightarrow Subject$ $TID \twoheadrightarrow Hobby$

and the 4NF decomposition is:

TEACHER_SUBJECT(TID,Subject) TEACHER_HOBBY(TID,Hobby)

This removes the redundancy caused by independent multivalued attributes.

Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

5NF – Project-Join Normal Form

Consider the relation:

SPJ(Supplier, Part, Project)

A supplier can supply different parts to different projects.

Example:

Supplier Part Project

S1 P1 J1

S1 P1 J2

S1 P2 J1

S1 P2 J2

Here, the same Supplier, Part, and Project information is repeated in several tuples. This creates redundancy.

Join Dependency

Suppose the following relationships exist independently:

- Supplier–Part
- Supplier–Project
- Part–Project

Then the relation has the join dependency:

$\ast(\text{SP}, \text{SJ}, \text{PJ})$

where:

$\text{SP}(\text{Supplier}, \text{Part})$ $\text{SJ}(\text{Supplier}, \text{Project})$ $\text{PJ}(\text{Part}, \text{Project})$

The original relation can be reconstructed by joining these three relations:

$\text{SPJ} = \text{SP} \bowtie \text{SJ} \bowtie \text{PJ}$

This is a join dependency.

Decomposition into 5NF

Decompose SPJ into three relations.

1. SUPPLIER_PART

Supplier Part

S1 P1

S1 P2

2. SUPPLIER_PROJECT

Supplier Project

S1 J1

S1 J2

3. PART_PROJECT

Part Project

P1 J1

P1 J2

P2 J1

P2 J2

Joining them reconstructs the original relation:

$SUPPLIER_PART \bowtie SUPPLIER_PROJECT \bowtie PART_PROJECT = SPJ$

Conclusion

The original SPJ(Supplier, Part, Project) relation contains redundancy because information about pairwise relationships is repeated across multiple tuples.

By decomposing it into:

SP(Supplier,Part) SJ(Supplier,Project) PJ(Part,Project)

the redundancy caused by the join dependency is removed.

Therefore, the decomposed relations are in 5NF (Project-Join Normal Form).

Key exam point: A relation is in 5NF when every non-trivial join dependency is implied by its candidate keys.

Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Consider a poorly designed college relation:

COLLEGE(SID, SName, DeptID, DeptName, CID, CName, Instructor)

Suppose the functional dependencies are:

$SID \rightarrow SName, DeptID$ $DeptID \rightarrow DeptName$ $CID \rightarrow CName, Instructor$

and a student can enroll in many courses.

The candidate key is:

(SID,CID)

because both student and course information are needed to uniquely identify an enrollment.

Anomalies

Insertion anomaly: We cannot add a new course until at least one student enrolls in it. Similarly, adding a department may require inserting unnecessary student information.

Update anomaly: If a department name changes, it may appear in many rows. Every occurrence must be updated. If one row is missed, inconsistent data is created. The same problem occurs for course names and instructor details.

Deletion anomaly: If the last student enrolled in a course is deleted, the course information may also be lost. Similarly, deleting the last student from a department may accidentally remove department information.

Normalization to 3NF

First separate student information:

STUDENT(SID, SName, DeptID)

Here:

$SID \rightarrow SName, DeptID$

Next separate department information:

DEPARTMENT(DeptID, DeptName)

where:

$DeptID \rightarrow DeptName$

Separate course information:

COURSE(CID, CName, Instructor)

where:

$CID \rightarrow CName, Instructor$

Finally, store enrollment separately:

ENROLLMENT(SID, CID)

with candidate key:

(SID, CID)

Final 3NF Schema

STUDENT(SID,SName,DeptID)

DEPARTMENT(DeptID,DeptName)

COURSE(CID,CName,Instructor) ENROLLMENT(SID,CID)

This decomposition removes insertion, deletion, and update anomalies and brings the database into **Third Normal Form (3NF)**.

Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

To compute attribute closure and candidate keys, we need a specific relation and set of functional dependencies. Here is a standard exam-style example.

Consider:

$R(A,B,C,D,E)$

with functional dependencies:

$A \rightarrow B \quad B \rightarrow C \quad CD \rightarrow E \quad E \rightarrow A$

Now compute closures.

1. Closure of A

Start with:

$A^+ = \{A\}$

Using $A \rightarrow B$:

$A^+ = \{A,B\}$

Using $B \rightarrow C$:

$A^+ = \{A,B,C\}$

No dependency gives us D, so:

$A^+ = \{A,B,C\}$

Therefore, A is not a candidate key.

2. Closure of E

Start with:

$$E^+ = \{E\}$$

Using $E \rightarrow A$:

$$E^+ = \{E, A\}$$

Then:

$$A \rightarrow B$$

and:

$$B \rightarrow C$$

So:

$$E^+ = \{E, A, B, C\}$$

Again, D is missing, so E is not a candidate key.

3. Closure of AD

$$(AD)^+ = \{A, D\}$$

From $A \rightarrow B$:

$$\{A, B, D\}$$

From $B \rightarrow C$:

$$\{A, B, C, D\}$$

Now $CD \rightarrow E$:

$$\{A, B, C, D, E\}$$

Therefore:

$$(AD)^+=\{A,B,C,D,E\}$$

So AD is a superkey. Since neither A nor D alone is a superkey:

AD is a candidate key

4. Closure of ED

$$(ED)^+=\{E,D\}$$

From $E \rightarrow A$:

$$\{E,D,A\}$$

From $A \rightarrow B$:

$$\{E,D,A,B\}$$

From $B \rightarrow C$:

$$\{E,D,A,B,C\}$$

Thus:

$$(ED)^+=\{A,B,C,D,E\}$$

Hence:

ED is also a candidate key

Final Answer

The important closures are:

$$A^+=\{A,B,C\} \quad E^+=\{A,B,C,E\} \quad (AD)^+=\{A,B,C,D,E\} \quad (ED)^+=\{A,B,C,D,E\}$$

Therefore, the candidate keys are:

AD, ED

Rule: An attribute set is a candidate key if its closure contains every attribute in the relation and no proper subset of it does. To compute attribute closure and candidate keys, we need a specific relation and set of functional dependencies. Here is a standard exam-style example.

Consider:

$R(A,B,C,D,E)$

with functional dependencies:

$A \rightarrow B \quad B \rightarrow C \quad CD \rightarrow E \quad E \rightarrow A$

Now compute closures.

1. Closure of A

Start with:

$A^+ = \{A\}$

Using $A \rightarrow B$:

$A^+ = \{A, B\}$

Using $B \rightarrow C$:

$A^+ = \{A, B, C\}$

No dependency gives us D, so:

$A^+ = \{A, B, C\}$

Therefore, A is not a candidate key.

2. Closure of E

Start with:

$E^+ = \{E\}$

Using $E \rightarrow A$:

$$E^+ = \{E, A\}$$

Then:

$$A \rightarrow B$$

and:

$$B \rightarrow C$$

So:

$$E^+ = \{E, A, B, C\}$$

Again, D is missing, so E is not a candidate key.

3. Closure of AD

$$(AD)^+ = \{A, D\}$$

From $A \rightarrow B$:

$$\{A, B, D\}$$

From $B \rightarrow C$:

$$\{A, B, C, D\}$$

Now $CD \rightarrow E$:

$$\{A, B, C, D, E\}$$

Therefore:

$$(AD)^+ = \{A, B, C, D, E\}$$

So AD is a superkey. Since neither A nor D alone is a superkey:

AD is a candidate key

4. Closure of ED

$$(ED)^+=\{E,D\}$$

From $E \rightarrow A$:

$$\{E,D,A\}$$

From $A \rightarrow B$:

$$\{E,D,A,B\}$$

From $B \rightarrow C$:

$$\{E,D,A,B,C\}$$

Thus:

$$(ED)^+=\{A,B,C,D,E\}$$

Hence:

ED is also a candidate key

Final Answer

The important closures are:

$$A^+=\{A,B,C\} \quad E^+=\{A,B,C,E\} \quad (AD)^+=\{A,B,C,D,E\} \quad (ED)^+=\{A,B,C,D,E\}$$

Therefore, the candidate keys are:

AD, ED

Rule: An attribute set is a candidate key if its closure contains every attribute in the relation and no proper subset of it does.